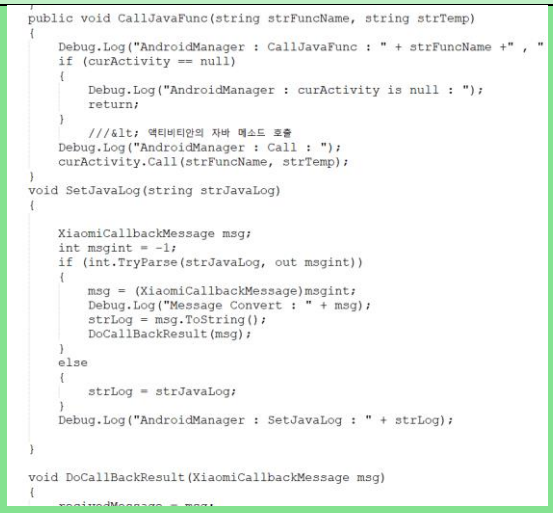
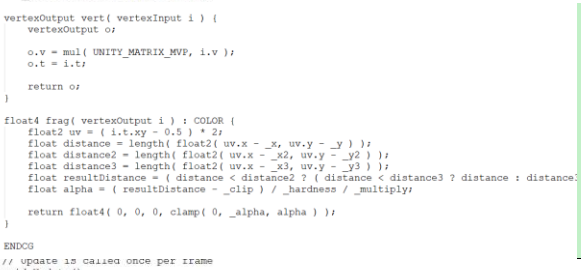
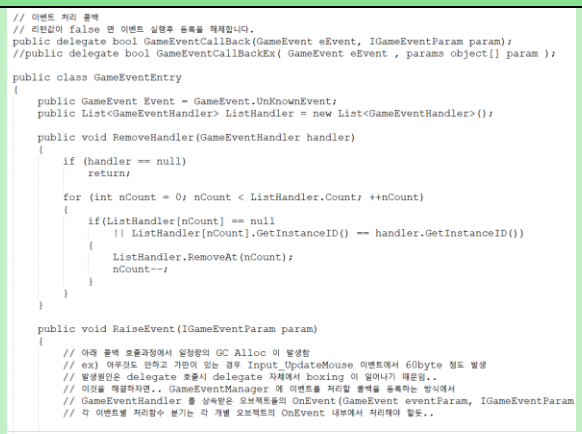
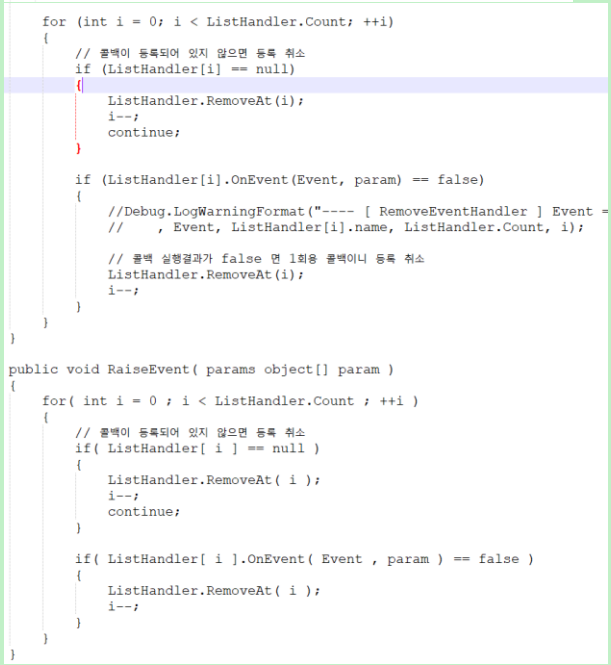
프로젝트 진행 내역		
이름	역할	샘플
TheMansion (싱글)	- 게임 전체 UI 구현(Android, IOS, Xbox), NGUI - 꼭두각시 동작 알고리즘 구현 - In-app 상품 데이터화 작업 - Com2us hub 모듈 삽입 및 데이터 연계화 - IOS 포팅 - 게임내 배치된 아이템 사용 알고리즘 구현 - 말풍선 동작 알고리즘 구현 - 인트로 영상 등의 action script 동작 구현 - 로컬라이징 (데이터 및 폰트 생성)	 void DirectionMove(int direction) { if (targetDestination != Vector3.zero) { if (direction == -1) {//1 SetRoomIncludeEnemy(); RoomManager rm = RoomManager.GetInstance; GameContext gc = GameContext.GetInstance; int leftX = roomIncludeDoll.positionX - 1; if (leftX <= 0) leftX = 0; int leftRoomID = gc.stage.rooms[leftX, roomIncludeDoll.positionY]; Room nextDollRoom = rm[leftRoomID]; if (roomIncludeDoll.positionX == 0) { Vector3 lastVal = TempmoveQueue.LastValue(); if (lastVal.x < 0.0f) lastVal = Vector3.zero; TempmoveQueue.Clear(); TempmoveQueue.Push(lastVal); moveHorizentalDir = 0; dollMoveEnabled = false; _state = WodenDollState.Idle; StopMove(direction); return; } else { if (nextDollRoom != null) { if (roomIncludeDoll.GetWallType(3) != RoomWallType.None || r public bool BuyItem(IAPItem itemType, uint buyNumber = 1) { SaveDataManager sdm = SaveDataManager.GetInstance; int money = sdm.GetCoinNumber(); if (money == 0) return false; //int price = itemData.GetItemPrice(itemType); int price = itemData.GetIAPGoodsPrice((IAPGoods)itemType); int payment = price * (int)buyNumber; if (money < payment) return false; GameContext gc = GameContext.GetInstance; sdm.UseCoin(payment); sdm.AddIAPitem(itemType, (int)buyNumber); sdm.Save(); return true; }
KinectDance (프로토타입)	- 게임 전체 UI 구현(PC),NGUI - 사진촬영 및 저장 데이터화 - 유저점수 통계 및 데이터화	코드 공유 불가

ProjectGD (프로토타입)	- TPS 캐릭터 이동 및 긴급회피 기능 구현 - Mecanim animation 의 mixing, blending 구현 - 적 캐릭터 AI 시스템 구현, 제3 정규화 - TPS 카메라 구현	 ``` void Update() { StageBase stage = GameContext.GetInstance.stage; //Debug.Log(heroAnimator.gameObject.name + " : " + myTrans.name); renewPos = Vector3.zero - heroAnimator.gameObject.transform.localPosition; heroAnimator.gameObject.transform.localPosition = Vector3.zero; myTrans.position += renewPos; if (stage.InGameState == StageBase.GameState.Win && myState != State.WIN) HeroFsm.SendEvent("WIN"); InputHero(); ChargeActivityGuage(); CalSoulGuageCount(); if (stage.InGameState != StageBase.GameState.Event) { UnbeatableCheck(); PickAttackTarget(); } if (GetCurrentEnemyTarget() != null) { Vector3 lookdir = GetCurrentEnemyTarget().GolemHitCollider[0].transform.position; GetTransform().LookAt(new Vector3(lookdir.x, 0, lookdir.z)); } } ```
ConsoleToMobile	- 함대전 관련 부분 구현 - 이벤트씬의 카메라 움직임 구현	코드 공유 불가
Xiaomi 포팅 (라이브)	- DefenseTechnica, TheMansion 의 Xiomi version 포팅작업. - 기존 프로젝트에서 Scaleform 제거, DFGUI 로 UI 재구축, - 중문화 작업 - Xiaomi Plugin 적용작업	 ``` public void CallJavaFunc(string strFuncName, string strTemp) { Debug.Log("AndroidManager : CallJavaFunc : " + strFuncName +" , " if (curActivity == null) { Debug.Log("AndroidManager : curActivity is null : "); return; } ///< 액티비티안의 자바 메소드 호출 Debug.Log("AndroidManager : Call : "); curActivity.Call(strFuncName, strTemp); } void SetJavaLog(string strJavaLog) { XiaomiCallbackMessage msg; int msgint = -1; if (int.TryParse(strJavaLog, out msgint)) { msg = (XiaomiCallbackMessage)msgint; Debug.Log("Message Convert : " + msg); strLog = msg.ToString(); DoCallBackResult(msg); } else { strLog = strJavaLog; } Debug.Log("AndroidManager : SetJavaLog : " + strLog); } void DoCallBackResult(XiaomiCallbackMessage msg) { ```

SpookyDoor (라이브)	- 파티클 이펙트 작업 - 유니티 셰이더 작업 (부분적으로 구멍내기) - 각종 버그 대응 - 인트로 애니메이션 제작 작업 - 인게임 배경 동작 - 가챠 UI 시스템 및 캐릭터 선택 등 UI 시스템	 
SexyJump (라이브)	- 단독 개발 작업 (전체 작업) - 광고 모듈 작업 (AdColony, Vungle, UnityAds) - 결제 모듈 (Soomla) - SNS 모듈 (SoomlaProfile) - 클라이언트->서버 데이터 전송 - LWF(Light weight flash) 적용 작업 - 전체 시스템 작업 - 화면 캡쳐, SNS	

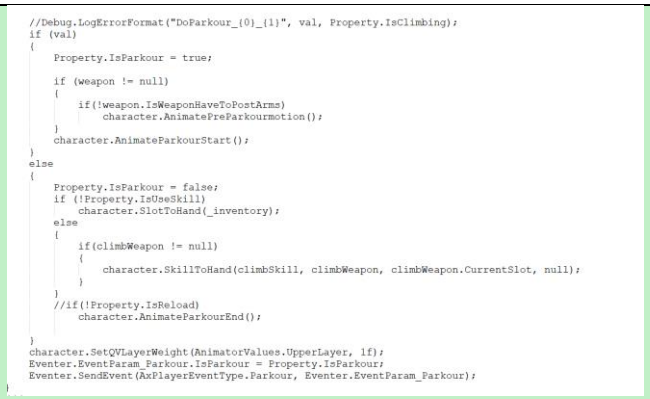
	공유 가능하도록 하는 작업 - 로컬라이징 작업.(데이터 작업) - AWS 이용한 유저간 랭킹 작업	
CFZero (라이브)	[FPS 모드] - FPS 캐릭터 시스템 및 Lagacy Animaiton 사용 시스템 작업 - FPS 캐릭터 동작 동기화작업 - FPS 캐릭터 사운드 작업 - AssetBundle 사용 및 암/복호화 작업 - 캐릭터 상/하체 각각 회전상태 동기화 - 번들 생성 자동화 작업 - 재장전 패킷에 대한 해킹 방지 작업 [BR모드] - TPS 캐릭터 시스템 및 Animator Controller 사용 시스템 작업 (Animator Value 전환에 의한	 

Animation State Change System)
- Parkour 시스템 및 동기화 작업
- TPS 캐릭터 동작 동기화 작업
- Ik 시스템 작업 (handIK)
- Weapon 시스템 및 Animation 작업
  차량 탑승시의 AimControll 작업
  [BRX모드]
- HyperFPS 캐릭터 시스템 및 Animator Controller 사용 시스템 작업 (StateMachine 에 의한 Animation State Call System)
- MultipleLayer Animator 사용하여 양손 분리되어 동작하도록 작업
- 캐릭터 스킬 시스템 및 동기화 작업
- StateMahine 시

```
public GameEvent Event = GameEvent.UnKnownEvent;
private readonly List<GameEventCallBack> _callBack = new List<GameEventCallBack>();
//public List<GameEventCallBackEx> CallBackEx = new List<GameEventCallBackEx>();

public void AddCallBack(GameEventCallBack callback)
{
    for (int nCount = 0; nCount < _callBack.Count; ++nCount)
    {
        if (_callBack[nCount] == callback)
            return;
    }

    _callBack.Add(callback);
}

public void RemoveCallBack(GameEventCallBack callback)
{
    for (int nCount = 0; nCount < _callBack.Count; ++nCount)
    {
        if (_callBack[nCount] == callback)
        {
            _callBack.RemoveAt(nCount);
            nCount--;
        }
    }
}

public void RaiseEvent(IGameEventParam param)
{
    for (int nCount = 0; nCount < _callBack.Count; ++nCount)
    {
        if (_callBack[nCount] == null)
        {
            _callBack.RemoveAt(nCount);
            nCount--;
            continue;
        }

        if (_callBack[nCount](Event, param) == false)
        {
            // 콜백 실행결과가 false 면 1회용 콜백이니 등록 취소
            _callBack.RemoveAt(nCount);
            nCount--;
        }
    }
}
```

```
var bundleName = DataTableManager.ItemTable.GetItemBundleName(entry.Value.Index);

UnityEngine.Assertions.Assert.IsTrue(bundleName.HasContent());

//캐릭터 로드 시작.
Debug.Log("LoadCharacter.playerData:" + playerData.baseEntry.CharacterID);
//if (playerData != null) Debug.Log("LoadCharacter.playerData.pathEntry:" + playerData.pathEntry);
//Debug.LogError("LoadCharacter.playerData.pathEntry.Base:" + playerData.pathEntry.Base);
var request1 = ResourceLoader.Instance.Load<GameObject>(bundleName, playerData.pathEntry.Base);
while (!request1.IsComplete)
    yield return null;
//캐릭터 베이스 노드 로드
GameObject go = (GameObject)request1.LoadedAsset;
character = go.GetComponent<AxCharacter>();

//캐릭터 QV바디 로드
//if (playerData != null) Debug.Log("LoadCharacter.playerData.pathEntry.QV:" + playerData.pathEntry.QV);
var request = ResourceLoader.Instance.Load<GameObject>(bundleName, playerData.pathEntry.QV);
while (!request.IsComplete)
    yield return null;

go = (GameObject)request.LoadedAsset;
AxQVBody qvBody = go.GetComponent<AxQVBody>();
go.name = go.name.Remove(go.name.Length - 7).Trim();

//캐릭터 PV 손 로드
//if (playerData != null) Debug.Log("LoadCharacter.playerData.pathEntry.PVArm:" + playerData.pathEntry.PVArm);
request = ResourceLoader.Instance.Load<GameObject>(bundleName, playerData.pathEntry.PVArm);
while (!request.IsComplete)
    yield return null;
go = (GameObject)request.LoadedAsset;
AxPVHand pvHand = go.GetComponent<AxPVHand>();
go.name = go.name.Remove(go.name.Length - 7).Trim();
```

```
    yield return null;
    //플레이어 초기화 종료
    this.gameObject.name = Name;
    OnLoaded(qvBody, pvHand);

    //Debug.Log("LOADED CHARACTER DONE = " + Name);

    HitBox = GetComponentInChildren<CFA.World.Hit.AxHitBox>();
    HitBox.AttachHitBoxInfo(this);

    //Debug.Log("LOADED CHARACTER DONE = " + Name);

    character.A_Throw = Throw;
    character.A_Punch = Punch;

    initialize = true;
    //Debug.LogErrorFormat("플레이어 스킵 로딩 시작지점 Player={0}, Time={1}", Name, Time.realtimeSinceStartup);

    ///캐시본 아이템 일단 제거
    //스킵 아이템 세트
                var activeSkillId = playerData.baseEntry.ActiveSkillID;
    var activeModelId = DataTableManager.AX_SkillTable.GetModelID(activeSkillId);
    AxItem activeSkill = AxItemManager.CreateItem(activeSkillId, activeModelId, activeSkillId);
    DoInvenPickSkill(activeSkill, AxSkillType.Active);

    var ultimateSkillId = playerData.baseEntry.UltimateSkillID;
    var ultimateModelId = DataTableManager.AX_SkillTable.GetModelID(ultimateSkillId);
    AxItem ultimateSkill = AxItemManager.CreateItem(ultimateSkillId, ultimateModelId, ultimateSkillId);
    DoInvenPickSkill(ultimateSkill, AxSkillType.Ultimate);

    character.OnIdleStart(
        () => {
            ActMove(Property.MoveDir, false);

            if (Property.IsReload)
            {
                Property.IsReload = false;
            }

        });
}
else
{
    Debug.LogErrorFormat("Not found character item data from BR_Ingame_ItemTable. [{0}]", playerData.base
}
```

	스템 작업 - Effect 시스템 작업 - Emote 기능 및 동기화 작업 - 캐릭터 모션 행동 패킷에 의한 동기화 이동 위치 동기화 보조 작업 및 디버깅	 ``` //Debug.LogErrorFormat("DoParkour_{0}_{1}", val, Property.IsClimbing); if (val) { Property.IsParkour = true; if (weapon != null) { if(!weapon.IsWeaponHaveToPostArms) character.AnimatePreParkourmotion(); } character.AnimateParkourStart(); } else { Property.IsParkour = false; if (!Property.IsUseSkill) character.SlotToHand(_inventory); else { if(climbWeapon != null) { character.SkillToHand(climbSkill, climbWeapon, climbWeapon.CurrentSlot, null); } } //if(!Property.IsReload) character.AnimateParkourEnd(); } character.SetQVLayerWeight(AnimatorValues.UpperLayer, 1f); Eventer.EventParam_Parkour.IsParkour = Property.IsParkour; Eventer.SendEvent(AxPlayerEventType.Parkour, Eventer.EventParam_Parkour); } ```
Guardian Tales (라이브)	- 캐릭터 및 액션 구현 작업 - 몬스터 AI 및 액션 구현 작업 - 전투 관련 기믹 / 옵션 구현 작업 - 신규 전투 컨텐츠 플로우 작업 - 파트장 롤(일정 확정 및 시스템 사양분석 등)	``` /// <summary> /// 메인 시스템에서 업데이트 실행. /// </summary> public void Update() { //신규 요청 추가 foreach (var item in requestCallbacks) { List<SubscribeCallbackInfo> callbacks = null; if (subscribeCallbacks.TryGetValue(item.Key, out callbacks)) { callbacks.Add(item.Value); } else { callbacks = new List<SubscribeCallbackInfo>(); callbacks.Add(item.Value); subscribeCallbacks.Add(item.Key, callbacks); } } // 다 더해줬으니 제거 requestCallbacks.Clear(); // 구독 삭제 foreach (var item in unSubscribeCallbacks) { List<SubscribeCallbackInfo> callbacks = null; { if (subscribeCallbacks.TryGetValue(item.Key, out callbacks)) { SubscribeCallbackInfo info = new SubscribeCallbackInfo(item.Value); if (callbacks.Remove(info)) { if (callbacks.Count == 0) { subscribeCallbacks.Remove(item.Key); } } } } } // 다 빼주면 제거 unSubscribeCallbacks.Clear(); ```

```
// 이번 프레임용
for (int i = 0; i < publishedOnThisFrame.Count; i++)
{
    List<SubscribeCallbackInfo> callbacks = null;
    PublishedEvent publishedEvent = publishedOnThisFrame[i];
    if (publishedEvent.target == null)
    {

        if (subscribeCallbacks.TryGetValue(publishedEvent.e.GetType(), out callbacks))
        {
            for (int j = 0; j < callbacks.Count; j++)
            {
                var callback = callbacks[j];
                bool removed = false;

                foreach (var unSubs in unSubscribeCallbacks)
                {
                    if (unSubs.Key == publishedEvent.e.GetType() && unSubs.Value == callback.Callback)
                    {
                        removed = true;
                        break;
                    }
                }

                if (removed)
                {
                    continue;
                }

                callback.Callback.Invoke(publishedEvent.e);

                //일회성 구독 삭제
                if (callback.SubscribeOnce)
                {
                    callbacks.RemoveAt(j--);
                }
            }
        }
    }
    else
    {
        // 이벤트 실행
        publishedEvent.target.OnEvent(publishedEvent.e);
    }

    publishedEvent.e.Dispose();
}
// 이번 프레임 종료함.
publishedOnThisFrame.Clear();
```

```
private static ObjectPool<ControllerPlayerState> pool = new ObjectPool<ControllerPlayerState>();
public static ControllerPlayerState Create(Character character)
{
    var s = pool.GetOrCreate();
    s.character = character;

    return s;
}

void IDisposable.Dispose()
{
    pool.Return(this);
}

private bool moveleft = false;
private bool moveright = false;

void ILateUpdatable.LateUpdateFrame(float dt)
{
    if (moveleft)
    {
        MoveLogic.ExcuteMove(character, Vector2.left, character.MoveSpeed * dt);
        character.HittableBounds.center = character.Position;
    }

    if (moveright)
    {
        MoveLogic.ExcuteMove(character, Vector2.right, character.MoveSpeed * dt);
        character.HittableBounds.center = character.Position;
    }


}

void IState.Enter(IState prevState)
{
    MessageSystem.Instance.Subscribe<TouchEvent>(((IEventListner)this).OnEvent);
}

void IState.Exit(IState nextState)
{
    MessageSystem.Instance.Unsubscribe<TouchEvent>(((IEventListner)this).OnEvent);
}
```